

README

基于 DDH 假设的私有交集求和协议

一、实验目的

实现并验证一个安全的 Private Intersection Sum with Cardinality 协议，使两个拥有不同用户数据的参与方可以在不泄露原始数据的前提下，计算出交集中用户的值总和及交集大小。

我们先来进行协议的复述：

输入：

- 双方：
 - 一个素数阶的群 $\mathcal{G}$ 和一个标识符空间 $\mathcal{U}$ 。存在一个哈希函数 $H : \mathcal{U} \rightarrow \mathcal{G}$ ，被建模为随机预言机，用于将标识符映射到群元素。
- P_1 ：集合 $X = \{v_i\}_{i=1}^{m_1}$ ，其中 $v_i \in \mathcal{U}$
- P_2 ：键值对集合 $W = \{(w_j, t_j)\}_{j=1}^{m_2}$ ，其中 $w_j \in \mathcal{U}$ ， $t_j \in \mathbb{Z}^+$

初始化设置：

- P_1 在群 $\mathcal{G}$ 中选择一个随机的私钥 k_1 。
- P_2 生成加法同态加密方案的密钥对 $(pk, sk) \leftarrow \text{AGen}(\lambda)$ ，并将公钥 pk 发送给 P_1 。

第 1 轮 (P_1)：

1. 对于集合中每一个元素 v_i ， P_1 计算哈希函数并用私钥 k_1 取幂，即计算：

$$H(v_i)^{k_1}$$

2. P_1 将集合 $\{H(v_i)^{k_1}\}_{i=1}^{m_1}$ 的乱序版本发送给 P_2 。

第 2 轮 (P_2)：

1. 对于从 P_1 接收到的每个元素 $H(v_i)^{k_1}$ ， P_2 用自己的密钥 k_2 再次取幂，得到：

$$Z = \{H(v_i)^{k_1 k_2}\}_{i=1}^{m_1}$$

2. P_2 将集合 Z 的乱序版本发送回 P_1 。

3. 对于自身输入集合中的每个 (w_j, t_j) ：

- P_2 对 w_j 进行哈希并用密钥 k_2 取幂，得到：

$$H(w_j)^{k_2}$$

- 使用公钥 pk 对 t_j 进行加密，得到：

$$\text{AEnc}(t_j)$$

4. P_2 将集合 $\{(H(w_j)^{k_2}, \text{AEnc}(t_j))\}_{j=1}^{m_2}$ 的乱序版本发送给 P_1 。

第 3 轮 (P₁) :

1. 对于从 P₂ 接收到的每个元素 $(H(w_j)^{k_2}, \text{AEnc}(t_j))$, P₁ 使用密钥 k_1 对第一个值再取幂, 得到:

$$H(w_j)^{k_1 k_2}$$

2. P₁ 计算交集索引集 J , 定义如下:

$$J = \{j : H(w_j)^{k_1 k_2} \in Z\}$$

其中 Z 是在第 1 轮中从 P₂ 收到的集合。

3. 对于交集中所有的 j , P₁ 将对应的密文相加, 计算出加密的交集求和密文:

$$\text{AEnc}(pk, S_J) = \text{ASum}(\{\text{AEnc}(t_j)\}_{j \in J}) = \text{AEnc}\left(\sum_{j \in J} t_j\right)$$

4. P₁ 使用 ARefresh 对密文进行随机化处理, 并将其发送给 P₂。
-

输出 (P₂) :

- P₂ 使用私钥 sk 解密第 3 轮收到的密文, 得到交集求和结果 S_J 。
-

二、协议参与方说明

该协议为一个两方计算协议, 参与者包括:

- Party 1 (P1):** 拥有一组标识符集合 $V = \{v_1, v_2, \dots, v_m\}$;
- Party 2 (P2):** 拥有另一组标识符与其对应值的集合 $W = \{(w_1, t_1), (w_2, t_2), \dots, (w_n, t_n)\}$, 其中 t_i 为整数值 (如消费金额、评分等)。

目标是:

- 双方在不泄露私有数据的前提下, 计算出交集大小 $C = |V \cap W|$ 与交集值的总和 $S = \sum_{i: w_i \in V} t_i$ 。
-

三、使用的密码学工具

- 哈希函数 $H(x)$: 模拟随机预言机, 将标识符映射到大整数;
 - 同态加密: 采用 Paillier 加密方案, 支持加法;
 - Diffie-Hellman 安全假设: 用于构造私有匹配机制
-

四、协议详细流程

预备阶段:

- 协议使用一个大素数 p 构造循环群 G , 模 p 进行所有幂运算;
- P2 生成 Paillier 加密密钥对 (pk, sk) , 将公钥 pk 发给 P1。

首先是协议的参数设置:

```
# ----- 协议参数设置 ----- #

# 一个大素数 p, 用于模拟群 G (DDH 假设所在群)
p = 2 ** 521 - 1

# 将标识符哈希到群元素
def hash_to_group(identifier: str) -> int:
    h = sha256(identifier.encode()).digest() # 进行 SHA256 哈希
    return int.from_bytes(h, 'big') % p      # 转为整数并取模 p

# 打乱数据并返回原始索引
def shuffle_with_indices(data):
    indices = list(range(len(data)))
    random.shuffle(indices)
    return [data[i] for i in indices], indices

# 反打乱数据
def unshuffle(data, indices):
    unshuffled = [None] * len(data)
    for i, idx in enumerate(indices):
        unshuffled[idx] = data[i]
    return unshuffled
```

之后是双方参与者定义:

```
# ----- 双方参与者类定义 ----- #

class Party1:
    """模拟 P1, 持有纯标识符集合"""
    def __init__(self, identifiers: List[str]):
        self.v = identifiers          # P1 的输入集合
        self.k1 = secrets.randbelow(p - 1) + 1 # 随机生成私钥 k1
        self.z = None                 # 存放 P2 返回的 z
        self.public_key = None        # 同态加密公钥
        self.intersection_indices = []

    def receive_public_key(self, pk):
        self.public_key = pk # 获取 Paillier 公钥

    def round1_send_hashed(self):
        # 对每个标识符进行哈希并做幂运算  $H(v_i)^{k1}$ 
        self.v_hashed = [pow(hash_to_group(v), self.k1, p) for v in self.v]
```

```

        self.v_hashed, self.shuffle_idx = shuffle_with_indices(self.v_hashed) #
打乱顺序
    return self.v_hashed

    def round3_compute_intersection_sum(self, data_from_p2: List[Tuple[int,
paillier.EncryptedNumber]]):
    # 接收 P2 的密文数据，找出与 Z 相同的元素，并同态求和
    result = []
    for w_hashed, enc_t in data_from_p2:
        hw = pow(w_hashed, self.k1, p)
        if hw in self.Z:
            result.append(enc_t)
    self.intersection_size = len(result)
    if result:
        s = result[0]
        for r in result[1:]:
            s += r # 同态加法
        return s
    return self.public_key.encrypt(0) # 如果交集为空，返回加密的 0

    def receive_Z(self, Z):
        self.Z = Z # 接收来自 P2 的  $Z = H(v_i)^{k_1 k_2}$ 

class Party2:
    """模拟 P2，持有标识符 + 值对"""
    def __init__(self, items: List[Tuple[str, int]]):
        self.w = items
        self.k2 = secrets.randbelow(p - 1) + 1 # 私钥 k2
        self.public_key, self.private_key = paillier.generate_paillier_keypair()
    # 同态加密密钥对

    def get_public_key(self):
        return self.public_key

    def round2_process_and_send(self, hashed_from_p1: List[int]) ->
Tuple[List[int], List[Tuple[int, paillier.EncryptedNumber]]]:
    # 计算  $H(v_i)^{k_1 k_2}$ 
    Z = [pow(h, self.k2, p) for h in hashed_from_p1]

    # 对自己输入 (wi, ti) 做处理，计算  $H(w_i)^{k_2}$  和加密的 ti
    processed = []
    for w, t in self.w:
        hw = pow(hash_to_group(w), self.k2, p)
        ct = self.public_key.encrypt(t) # 同态加密
        processed.append((hw, ct))

    processed, _ = shuffle_with_indices(processed) # 打乱顺序
    return Z, processed

    def round3_decrypt_sum(self, encrypted_sum: paillier.EncryptedNumber):
        return self.private_key.decrypt(encrypted_sum) # 解密最终结果

```

Round 1: P1 执行

1. P1 选择一个私钥 $k_1 \in \mathbb{Z}_p^*$;
2. 对于每个标识符 $v_i \in V$, 计算 $h_i = H(v_i)^{k_1} \bmod p$;
3. 将所有 h_i 打乱顺序后发送给 P2。

P1 发送给 P2: 打乱后的集合 $\{H(v_i)^{k_1}\}$ 。

Round 2: P2 执行

1. P2 选择私钥 $k_2 \in \mathbb{Z}_p^*$;
2. 对收到的每个 h_i , 计算 $h_i^{k_2} = H(v_i)^{k_1 k_2}$, 组成集合 Z ;
3. 对自己的数据 (w_j, t_j) :
 - 计算 $H(w_j)^{k_2} \bmod p$;
 - 加密 t_j 得到 $\text{Enc}(t_j)$;
 - 发送打乱后的 $(H(w_j)^{k_2}, \text{Enc}(t_j))$ 给 P1。

P2 发送给 P1:

- 集合 $Z = \{H(v_i)^{k_1 k_2}\}$;
- 加密数据集 $\{(H(w_j)^{k_2}, \text{Enc}(t_j))\}$ 。

Round 3: P1 执行

1. 对每个 $(H(w_j)^{k_2}, \text{Enc}(t_j))$, 计算 $H(w_j)^{k_1 k_2}$;
2. 若该值在集合 Z 中, 则对应 $\text{Enc}(t_j)$ 加入结果集;
3. 对所有交集密文值进行 Paillier 同态加法, 得到总和密文 $\text{Enc}(S)$;
4. 将其发送给 P2。

P1 发送给 P2: $\text{Enc}(S)$ (交集的值之和)。

输出阶段: P2 执行

P2 使用私钥解密密文 $\text{Enc}(S)$, 得到交集的总和 S 。
P1 同时可计算出交集的大小 C 。

详细代码:

```
# ----- 协议运行模拟 ----- #

def run_ddh_psi_sum():
    # 输入数据
    p1_input = ['alice', 'bob', 'carol', 'dave']
    p2_input = [('bob', 10), ('carol', 20), ('eve', 30)]
    # P1 拥有标识符
    # P2 拥有标识符
    +值对
```

```
# 初始化两方
P1 = Party1(p1_input)
P2 = Party2(p2_input)

# Round 0: 公钥交换
pk = P2.get_public_key()
P1.receive_public_key(pk)

# Round 1: P1 对每个标识符哈希并加密后发送
round1_msg = P1.round1_send_hashed()

# Round 2: P2 接收并返回 Z 和加密数据
Z, enc_data = P2.round2_process_and_send(round1_msg)
P1.receive_Z(Z)

# Round 3: P1 找交集并求和（加密）
encrypted_sum = P1.round3_compute_intersection_sum(enc_data)

# 最终由 P2 解密得到结果
total = P2.round3_decrypt_sum(encrypted_sum)

print("交集求和值 (Intersection sum):", total)
print("交集大小 (Intersection size):", P1.intersection_size)
```

五、输出结果

- P1 得到：交集大小 $C = |V \cap W|$;
- P2 得到：交集总值 $S = \sum_{i:w_i \in V} t_i$ 。

六、安全性分析

该协议在 **半诚实模型 (semi-honest adversary)** 下是安全的：

- **基于 DDH 假设**：任何一方无法反推出原始哈希前的标识符；
- **同态加密语义安全**：即便密文被观察，也无法获取 t_j 的具体值；
- **打乱顺序 (Shuffle) 保护隐私**：避免泄露标识符之间的对应关系。

七、实验实现说明

- 实现语言：Python 3；
- 同态加密库：`phe`（实现 Paillier 加密）；
- 哈希函数：使用 `SHA-256`，并将输出转为大整数；
- 打乱顺序使用 Python 内置 `random.shuffle()`；
- 输入样例：

P1 输入: ['alice', 'bob', 'carol', 'dave']
P2 输入: [('bob', 10), ('carol', 20), ('eve', 30)]

最终输出为:

交集求和值 (Intersection sum): 30
交集大小 (Intersection size): 2

解释: 交集为 {'bob', 'carol'}, 对应的值之和为 $10 + 20 = 30$ 。

```

D:\crypto-projects\DDH-based Private Intersection-Sum\.venv\Scripts\python.exe "D:\crypto-projects\DDH-based Private Intersection-Sum\main.py"
交集求和值 (Intersection sum): 30
交集大小 (Intersection size): 2

进程已结束，退出代码为 0

```

八、总结与拓展

该协议具备如下优点:

- 无需可信第三方即可安全计算;
- 有良好的可扩展性 (支持分段、多值、隐私增强等) ;
- 实现简单, 适合实际部署于云端或多方计算系统。